Flow-SLM: Joint Learning of Linguistic and Acoustic Information for Spoken Language Modeling

Ju-Chieh Chou
Toyota Technological Institute at Chicago
IL, USA
jcchou@ttic.edu

Jiawei Zhou
Stony Brook University
NY, USA
jiawei.zhou.1@stonybrook.edu

Karen Livescu
Toyota Technological Institute at Chicago
IL, USA
klivescu@ttic.edu

***Abstract*—Textless spoken language models (SLMs) are generative models of speech that do not rely on text supervision. Most textless SLMs learn to predict the next semantic token, a discrete representation of linguistic content, and rely on a separate vocoder to add acoustic information to the generated speech. Such models have no access to acoustic context and no built-in control over acoustic details. In this work, we propose to jointly model linguistic and acoustic information by generating semantic tokens and a continuous real-valued representation of the acoustic frame. We use a flow-matching objective to predict the continuous vector conditioned on the semantic tokens. We study the design space of this approach and find that predicting multiple future semantic tokens helps preserve linguistic information. Our approach achieves comparable performance to existing models in terms of linguistic likelihood benchmarks, while providing better acoustic detail in prompted generation.[1]**

***Index Terms*—Spoken language modeling, textless spoken language modeling, flow matching.**

I. Introduction

Speech carries both linguistic information—related to the content in the spoken word sequence—and acoustic information, such as the speaker identity, prosody,[2] and noise conditions. Acoustic information plays an important role both in the naturalness of the signal and by conveying extra-linguistic information such as emotion.

Textless spoken language models (SLMs) [1]–[4] aim to directly learn generative models of speech without using text as an intermediate representation. Existing approaches primarily focus on generating "semantic tokens" that encode linguistic content, while relegating the generation of acoustic information to a separate conditional vocoder.

In this work, we develop and investigate a new approach to jointly learn from and generate both linguistic and acoustic signals. Joint modeling enables access to and control over both linguistic and acoustic information, allowing the model to capture fine-grained acoustic nuances while maintaining the intended linguistic content.

One approach to representing speech for faithful acoustic generation is to use residual vector quantization (RVQ) [5]–[7], which tokenizes speech into multiple levels of detail. In this work, we propose to directly generate a real-valued vector at each timestep, using a model learned with a conditional flow matching (CFM) objective [8]. We first generate semantic tokens, and then use a CFM prediction head conditioned on the context and the semantic tokens to predict the current real-valued vector. The generated vectors are then transformed into a waveform by a vocoder. While the semantic tokens capture linguistic content, the CFM head generates fine-grained acoustic detail. In contrast to predicting multiple hierarchical discrete tokens, CFM uses a simple regression objective during training, substantially reducing overhead and memory consumption (see Sec. III-D).

We find that simply replacing discrete tokens with continuous-valued tokens compromises the generation of meaningful linguistic content, because the model learns to focus on local frame-level continuity rather than learning high-level semantic structure. Instead, we add a prediction head for N future timesteps and find that this approach can mitigate this problem while retaining acoustic detail.

Our experimental results show that Flow-SLM achieves comparable performance in terms of semantic metrics to existing discrete-token-only models, while being trained on less compute and data. Flow-SLM is also more acoustically faithful to the ground truth, in terms of distributional metrics and speaker preservation, and is capable of generating diverse acoustic attributes in prompted generation. Overall, our approach has promising performance simultaneously on multiple metrics of both linguistic and acoustic quality, using relatively modest compute and data resources.

II. Related work

A. Speech tokenization

Textless SLMs generally require speech tokenization to convert speech signals into discrete tokens. For example, GSLM [2] applies k-means clustering to HuBERT [9] representations to extract semantic tokens, which are designed to capture only the linguistic content of the speech rather

[1]Demo page: https://jjery2243542.github.io/flowslm.github.io/

[2]Strictly speaking, prosody can include both linguistic and non-linguistic information.

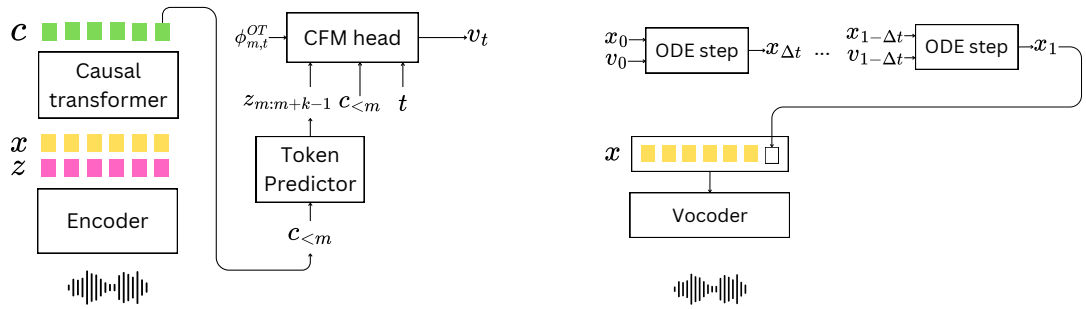

Fig. 1. *Left*: The Flow-SLM architecture (see Sec. III). The encoder maps the waveform into semantic tokens z and continuous embeddings x. The causal transformer maps the sequence of embeddings x into a context vector c. At each timestep m, the token predictor predicts the semantic tokens $z_{m:m+k-1}$ from the context vector $c_{<m}$. The CFM head predicts the vector field v_t from optimal transport conditional flow $\phi^{OT}_{m,t}$ (Sec. III), conditioning on the semantic tokens $z_{m:m+k-1}$ and the context vector $c_{<m}$. *Right*: The inference process per timestep for the CFM head. During inference, at each timestep, an embedding for the current timestep is generated with an ODE solver. The ODE solver iteratively takes x_t and the velocity from the CFM head (v_t) to generate the next $x_{t+\Delta t}$. After the whole embedding sequence is produced, it is decoded to a waveform by the vocoder.

than enable reconstruction. To address the lack of acoustic detail, [10] proposes adding speaker and prosody information through a conditional vocoder.

Another approach to tokenization is using a speech codec, which encodes speech into discrete tokens and uses a decoder to reconstruct the original waveform, thereby retaining acoustic detail by design. One commonly used type of codec is based on residual vector quantization (RVQ), used in SoundStream [5], Encodec [11], and others. RVQ quantizes the residuals of vector-quantized representations. This allows each frame to be tokenized into N levels of tokens, from coarse to fine. RVQ tokenizers generally yield higher speech quality than conditional vocoders that condition on the semantic tokens to generate waveforms, and the resulting tokens are commonly referred to as acoustic tokens. To integrate both semantic and acoustic information, SpeechTokenizer [7] and Mimi [6] distill semantic tokens into the first level of an RVQ quantizer. This approach produces a unified tokenizer that exploits semantic tokens for modeling linguistic content and acoustic tokens for high-fidelity reconstruction.

B. Spoken language models

Spoken language models (SLMs) [12] can be categorized into three main types: (1) speech-aware text LMs [13], [14], which can take speech and text as input but produce only text as output, (2) pure speech LMs, where both input and output are speech, which are trained without text [1]–[3], and (3) joint speech-text LMs, which model both speech and text input and output [6], [15], [16]. Our work falls into the second category.

In pure speech LMs, acoustic details in the generated speech are typically added using a conditional vocoder [2], [3], [10] or via hand-crafted features such as pitch [4], [16] to generate waveforms from a semantic token sequence. However, these approaches offer limited control over acoustic properties based on content and lack access to acoustic information in the input speech during generation.

Some joint speech-text LMs introduce text as an intermediate representation [6], [17]. These models simultaneously generate text and speech streams. Although this approach tends to produce more semantically coherent speech, it requires transcriptions to train the language models.

C. Conditional flow matching

Conditional flow matching (CFM) is an objective for learning generative models, typically for continuous data, which has been increasingly used in the last few years [8], [18]. CFM is one of several types of generative models (another being diffusion models [19], [20]) that generate by mapping samples from a simple prior distribution to samples from the desired distribution. CFM learns an ordinary differential equation (ODE), which we can sample from by moving along the learned vector field, parameterized by a neural network, from the prior distribution to the data distribution. CFM has been shown to reduce the number of function evaluations (NFE), the number of times the neural function is evaluated, compared to diffusion models. Since CFM learns directly from continuous distributions, it does not require a quantizer to learn a generative model. This makes it well-suited for modalities that are inherently continuous, such as image and audio.

CFM has been used for speech generation in several ways [21]–[25]. [22], [23] use CFM as a pre-training objective for generative speech representation models that can be fine-tuned on other tasks. [21], [24] use CFM as an objective for TTS. Our work is, to our knowledge, the first to apply CFM in the context of textless spoken language modeling.

III. Method

A. Background: Conditional flow matching (CFM)

In this section, we provide background on CFM [8] using unconditional generation as an example, but in our models we use a conditional version (Eq. 10), i.e. conditioning on the semantic tokens and the past context.

Suppose we have an unknown d-dimensional data distribution $q(x)$ that we would like to model. Flow matching (FM) formulates the problem as learning the *flow* from a known prior distribution $p_0(x)$ to a distribution $p_1(x)$ that approximates the

data distribution $q(x)$: $p_1(x) \approx q(x)$. The flow $\phi_t(x)$ is defined by an ODE:

$$\frac{d}{dt}\phi_t(x) = v_t(\phi_t(x)); \phi_0(x) = x, \tag{1}$$

where $v_t(\phi_t(x)) : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ is a time-dependent vector field. We can sample x by solving Eq. 1 from $t = 0$ to $t = 1$.

The goal of FM is to learn a parameterized neural function $v_t(x;\theta)$ to match the ground truth vector field $u_t(x)$, minimizing

$$L_{FM} = \mathbb{E}_{t,p_t(x)}[\|v_t(x;\theta) - u_t(x)\|^2]. \tag{2}$$

However, the ground truth vector field $u_t(x)$ is unknown. Fortunately, [8] shows that we can equivalently learn from a conditional, sample-specific vector field $u_t(x|x_1)$ with $x_1 \sim q(x)$, using the CFM loss:

$$L_{CFM}(\theta) = \mathbb{E}_{t,q(x_1),q(x|x_1)}\|v_t(x;\theta) - u_t(x|x_1)\|^2, \tag{3}$$

We use the optimal transport conditional flow [8]:

$$\phi_t^{OT}(x_0) = tx_1 + (1 - (1 - \sigma_{min})t)x_0, \tag{4}$$

with $x_1 \sim q(x_1), x_0 \sim p_0(x) = \mathcal{N}(0, \mathbf{I})$ and $\sigma_{min} = 10^{-5}$ following [23]. The corresponding conditional vector field is given by:

$$u_t(\phi_t^{OT}(x_0)|x_1) = x_1 - (1 - \sigma_{min})x_0. \tag{5}$$

Finally, the learned vector field $v_t(\cdot;\theta)$ is trained to approximate the conditional vector field:

$$\mathbb{E}_{t,q(x_1),p_0(x_0)}[\|v_t(\phi_t^{OT}(x_0);\theta) - (x_1 - (1 - \sigma_{min})x_0)\|_2^2]. \tag{6}$$

In summary, the CFM head takes a sample $\phi_t^{OT}(x_0)$ as input and outputs a prediction of the vector field $v_t(\cdot;\theta)$ to approximate the conditional vector field $u_t(\phi_t^{OT}(x_0)|x_1)$ during training.

B. Flow-SLM architecture and training

Our architecture consists of four components: an encoder, a causal transformer, a semantic token predictor, and a CFM head as shown in Fig. 1. For the encoder we use a pre-trained, frozen encoder, resulting in a set of learnable parameters $\theta = \{\theta_T, \theta_{sem}, \theta_{CFM}\}$ for the causal transformer, semantic token predictor, and CFM head respectively.

Given a distribution over speech waveforms $\mathcal{D}$, we sample a waveform of length L samples: $s_{1:L} \sim \mathcal{D}$. The encoder $\text{Enc}(\cdot)$ transforms the waveform into a sequence of M embeddings $x_{1:M} \in \mathbb{R}^{M \times d}$, and a semantic token sequence $z_{1:M}$ where each $z_m \in \mathcal{V}$ for some vocabulary $\mathcal{V}$:

$$x_{1:M}, z_{1:M} = \text{Enc}\left(s_{1:L}\right). \tag{7}$$

We use the pre-quantization embedding in Mimi [6], i.e. the output of the encoder, as our embedding x, and its first RVQ layer as semantic tokens z.

The causal transformer $\text{T}(\cdot)$ encodes the past context $x_{1:m-1}$ into a context vector $c_{<m}$:

$$c_{<m} = \text{T}\left(x_{1:m-1}\right). \tag{8}$$

The semantic token predictor $P_{sem-k}(\cdot)$ predicts the next k semantic tokens $z_{m:m+k-1}$ given the context vector $c_{<m}$, and is learned by minimizing a *multi-token* cross-entropy loss:

$$L_{sem-k}(\theta) = \frac{1}{k}\mathbb{E}[-\log P_{sem}(z_{m:m+k-1}|c_{<m})] \tag{9}$$

As we demonstrate in our experiments, predicting further into the future is crucial for achieving good performance in terms of content-related metrics with our model.

The CFM head is responsible for modeling the distribution of x_m at each timestep m, given the previous context and the predicted semantic tokens. Our flow matching head $v_t(\cdot;\theta_{CFM})$ is trained with a CFM objective as described in Sec. III-A, but with additional conditioning on the context vector $c_{<m}$ and the semantic tokens $z_{m:m+k-1}$:

$$\begin{aligned} &L_{CFM-k}(\theta) \\ &= \mathbb{E}_{t,q(x_1),p_0(x_0)} \quad [\|v_t(\phi_t^{OT}(x_0), c_{<m}, z_{m:m+k-1}) - u_t\|_2^2], \end{aligned} \tag{10}$$

where u_t is defined as in Eq. 5 and where we have dropped the subscript m in x_m for simplicity. Our final loss is

$$L(\theta) = L_{sem-k}(\theta) + L_{CFM-k}(\theta) \tag{11}$$

C. Flow-SLM inference

During inference, we first generate the context vector $c_{<m}$ based on the available context and sample the semantic token $z_{m:m+k-1}$. Then at each timestep m, we sample from the prior distribution $x_{m,0} \sim \mathcal{N}(0, \mathbf{I})$, then use the learned ODE produced by the CFM head, which is conditioned on $\{c_{<m}, z_{m:m+k-1}\}$, to transform $x_{m,0}$ into $x_{m,1} = x_m$. The generated embedding x_m is appended to the input of the causal transformer for predicting the following timestep $m+1$. After collecting the whole sequence until the end-of-sequence token, we decode the embedding sequence into a waveform using a vocoder (the Mimi decoder + quantizer) as shown in Fig. 1.

We note that Flow-SLMs require only one forward pass for the causal transformer, but multiple passes for the CFM head, to generate one token, saving the compute for the transformer.

D. Comparison between RVQ and CFM

Another approach for modeling acoustic details is to use RVQ tokens [6]. RVQ has tjevltbhetjjjvjkgtrknvjdltkkbvbntvlucitlddfriberjgcudlveerihvnvldhe disadvantage that it requires the multiple levels of acoustic tokens to be ordered in some way, and this can lead to a larger computation overhead when predicting them auto-regressively. For example, Mimi [6] represents speech as 12.5 real-valued vectors per second, each of which is quantized into 32 tokens ordered from coarse to fine. To model the hierarchical structure of these tokens during generation, they are arranged either by using a "delay pattern" [6] or by flattening them into a sequence [3].

Training with the CFM objective requires less memory than its RVQ counterpart. For example, in RVQ with token flattening, training with N levels of tokens requires memory of $B*T*N*|\mathcal{V}|$ per batch, where B is the batch size, T is the number of timesteps, and V is the vocabulary. On the other hand, flow matching requires running only one d-dimensional linear regression head per timestep ($B * T * d$).

IV. Experiments

A. Setup

Our main training data is the English subset of Multilingual LibriSpeech (MLS-En) [26], which contains approximately 45k hours of speech, corresponding to around 2 billion continuous tokens with the Mimi encoder [6]. We also use an extended training set, which includes these 45k hours and an additional 20k hours from the clean subset of The People's Speech [27] to expose the model to a non-audiobook domain. We use MLS-En (45k) as the base setup if not otherwise specified, and MLS-En plus The People's Speech clean (65k total) as the extended setup, denoted by "-ext".

We initialize the causal transformer from a text LM, following TWIST [1], specifically using two OpenELM [28] checkpoints (OpenELM-270M and OpenELM-1B) for our two model sizes. For the CFM head, we adopt the same architecture as in [29], using 3 residual blocks ($\sim$125M parameters) of MLP for our smaller model Flow-SLM-270M and 6 residual blocks ($\sim$150M parameters) for the larger Flow-SLM-1B.

The model is trained for one epoch with a batch size of 128 utterances. We use 8-bit AdamW [30] as the optimizer to reduce GPU memory usage. We use the Mimi encoder [6] to extract representations, with pre-quantizer outputs serving as the embeddings x and the first RVQ level used as the semantic tokens z. We use classifier-free guidance (CFG) [31] to sharpen the distribution, by randomly dropping the conditioning input to the CFM head $(z_{m:m+k-1}, c_{<m})$ with a probability of 0.05 during training.

During inference, we use nucleus sampling [32] with a top-p value of 0.95 when sampling semantic tokens. We find that Flow-SLMs tend to generate excessive silence, which we reduce by penalizing the logits of silence tokens during sampling.[3] We sample from the learned ODE using a midpoint ODE solver with NFE=64 based on preliminary experiments. We apply a temperature of 0.8 to the prior Gaussian distribution (which is equivalent to changing the variance of the prior) and set the CFG scale [31] to 0.3.

The generated embeddings are converted to waveforms using the Mimi quantizer with the first 16 RVQ levels (including the later levels introduces more artifacts) and decoder [6].

B. Evaluation

a) Semantic modeling: We evaluate semantic modeling performance using sWUGGY and sBLIMP from the ZeroSpeech 2021 challenge [34]. sWUGGY measures the model's ability to distinguish words from non-words: each testing pair consists of a word and a non-word, and the pair is considered correct if the model assigns a higher probability to the true word than to the non-word. We use all the vocabulary in the sWUGGY experiment. sBLIMP follows a similar setup but focuses on sentence grammar.

[3]Silence tokens are identified using VAD from the TorchAudio package [33], and we subtract 10 from their logits for decoding steps.

b) SALMon metrics: We measure acoustic consistency, semantic-acoustic alignment, and background alignment using the SALMon [35] benchmark. One important note is that we use the CFM head to generate acoustic details conditioned on the semantic tokens, and the CFM head does not provide an explicit likelihood to evaluate the acoustic aspect of the generated data. Therefore, we only use the semantic token predictor to measure likelihood in SALMon, which may not fully reflect the model's ability on acoustic tasks.

For evaluation of semantic-acoustic alignment, we use SALMon's sentiment alignment task. In this task, a "positive" utterance conveys the same sentiment in both the content and the prosody (for example, using a sad tone when saying "I am sad"), while a negative example is the same sentence with mismatched sentiment. For acoustic consistency, we average over all attributes in SALMon. Acoustic consistency includes pairs of positive and negative samples with the same content. Positive samples have consistent attributes, such as speaker, while negative samples are inconsistent at word boundaries.

c) Generated speech quality: To evaluate the quality of the generated speech, we perform prompt continuation using a randomly sampled subset of 500 utterances from the LibriSpeech test-clean and test-other sets [36]. We use the first 3 seconds of each utterance as a prompt and continue generation until 10 seconds. We sample 4 continuations for each prompt and use these continuations for all generation-related metrics. We then evaluate in several ways. First, we transcribe the speech using Whisper-large-v3-turbo [37] and evaluate the perplexity of the text using LLaMA-3.2-1B [38], a metric referred to as *genPPL* [1], [39].[4]

In addition, we measure the speaker similarity between the prompt and continuation, using a WavLM-large-TDNN [40] speaker verification model, which is fine-tuned from WavLM, following [22], [24], [41].[5]. For this purpose, we resynthesize the prompt with the Mimi decoder [6] to eliminate any inconsistency due purely to the vocoder.

Finally, we use the Fréchet speech distance (FSD) [24], [41] to measure the dissimilarity between the generated distribution and the ground-truth distribution for prompted generation. Specifically, we use two variants of FSD: e2v FSD uses the utterance-level representations from emotion2vec [42] to measure the expressiveness of the generated speech, following [41], and wlm FSD uses WavLM-large layer 6 (the layer is selected based on a previous study [43]) as features to measure the overall quality of the per-frame distribution.

V. Results

For all of the results below, unless otherwise specified, we train Flow-SLMs by predicting 4 future tokens (i.e., using the L_{sem-4} loss) and using them as a condition for the CFM head.

[4]Note that the text perplexity here measures only the fitness of the generated speech content to LlaMA-3.2-1B's distribution, which may not fully align with human preferences. We use the small LM for genPPL following prior work [1], [39], but a better language model may provide better evaluation.

[5]https://github.com/microsoft/UniSpeech/tree/main/downstreams/speaker_verification

TABLE I

ZEROSPEECH METRICS. THE TABLE IS DIVIDED INTO SECTIONS CORRESPONDING TO MODELS OF APPROXIMATELY THE SAME SIZE. *: AUDIOLM ONLY USES THE SEMANTIC TOKEN MODEL (300M PARAMETERS) FOR SWUGGY AND SBLIMP.

#	Model	sWUGGY↑	sBLIMP↑
1	TWIST-350M	69.7	56.2
2	AudioLM-300M*	71.5	64.7
3	Flow-SLM-270M	68.7	57.3
4	TWIST-1.3B	72.7	57.0
5	VoxtLM-1.3B	65.6	57.1
6	Flow-SLM-1B	69.8	60.0
7	Flow-SLM-1B-extend	65.7	60.9
8	Moshi-7B	74.3	58.9
9	SpiritLM-base-7B	69.0	58.3
10	SpiritLM-exp-7B	65.0	54.2

TABLE II

METRICS RELATED TO GENERATION. "WLM FSD" = WAVLM FSD, "SPK SIM" = SPEAKER SIMILARITY, "EXT." = EXTENDED. THE ABLATION STUDY (THE LAST THREE ROWS) ARE CONDUCTED ON THE DEV SET.

Model	genPPL↓	spk sim↑	e2v FSD↓	wlm FSD↓
ground truth	64.3	0.66		
TWIST-350M	116.7	0.09	6.16	1416.7
Flow-SLM-270M	173.3	0.36	3.23	1235.4
TWIST-1.3B	98.6	0.09	5.62	1417.4
Flow-SLM-1B	147.5	0.43	2.82	1018.1
Flow-SLM-1B ext.	137.6	**0.45**	**2.62**	**997.9**
SpiritLM-base-7B	**54.7**	0.11	5.31	1954.8
SpiritLM-exp-7B	84.0	0.11	3.85	1315.1
Flow-SLM-270M	178.9	0.37	4.70	1240.9
- future prediction	314.1	0.37	6.20	1007.1
- sem. token	407.1	0.42	5.56	1045.6

A. Comparison to other SLMs

We compare Flow-SLM to previous approaches, including TWIST [1], SpiritLM [16], Moshi [6], AudioLM [3], and VoxtLM [44] in Table I and Table II. First, we consider linguistic content-related metrics, including sWUGGY and sBLIMP in Table I and genPPL in Table II. We can see that Flow-SLM-1B lags behind TWIST-1.3B [1] in terms of sWUGGY, but achieves a better result on sBLIMP than all previous models except for audioLM. Flow-SLM-1B outperforms the larger SpiritLM-base-7B in terms of both sWUGGY and sBLIMP. When scaling up the data size, Flow-SLM-1B-extend has a slightly improved sBLIMP but a degraded sWUGGY. One possible explanation is that adding The People's Speech as training data introduces a distribution mismatch between the training data and the testing data in sWUGGY, as half of the vocabulary in sWUGGY is taken from the LibriSpeech training data.

However, in terms of genPPL, the Flow-SLM-1B family performs worse than the TWIST and SpiritLM [16] families, suggesting that Flow-SLMs are less coherent in terms of the generated linguistic content. Potential reasons include: (1) Flow-SLMs are trained with less compute and data (see Fig. 2), and (2) the joint learning of linguistic and acoustic information may compromise the modeling of longer-range semantics. An important direction for future work is therefore to analyze the scaling behavior of Flow-SLMs.

TABLE III

SALMON BENCHMARK ACCURACY (%). THE ACOUSTIC CONSISTENCY METRIC IS AN AVERAGE OF ALL ACOUSTIC CONSISTENCY CATEGORIES IN SALMON. "SENT. ALIGNMENT" STANDS FOR SENTIMENT ALIGNMENT, AND "BG. ALIGNMENT" STANDS FOR BACKGROUND ALIGNMENT. *: WE COUNT THE PARAMETERS BASED ON THE CHECKPOINT PROVIDED.

Model	Consist. ↑	sent. alignment ↑	bg. alignment ↑
TWIST-350M	62.5	51.5	56.5
Flow-SLM-270M	70.8	**60.0**	55.5
TWIST-1.3B	62.5	53.0	56.5
Flow-SLM-1B	**71.6**	57.0	56.0
Flow-SLM-1B-ext.	**71.6**	57.0	53.0
pGSLM-151M*	64.8	55.5	53.5
TWIST-7B	63.3	51.5	54.5
SpiritLM-exp-7B	69.0	52.0	**59.5**

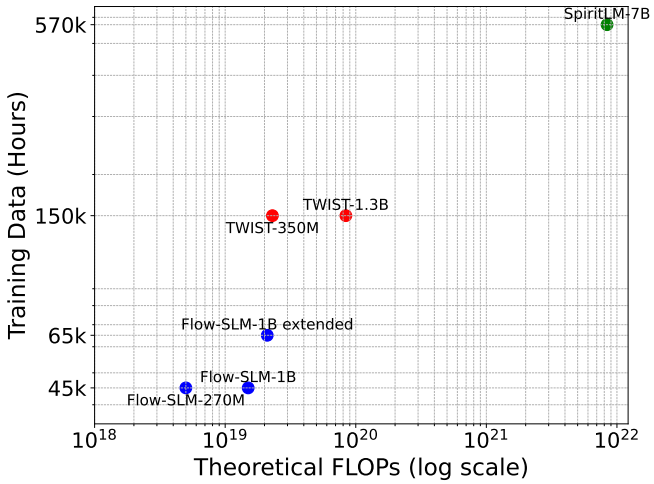

Fig. 2. Compute and data comparison across models. Following [45], we estimate the theoretical FLOPs as $6 * N_{param} * D_{tokens}$, where N_{param} is the number of training parameters excluding embeddings and D_{tokens} is the number of training tokens. For SpiritLM, we include the total amount of speech data, including both speech-only and speech-text data.

Next, we use the SALMon benchmark [35] to measure acoustic consistency, sentiment alignment, and background alignment, as shown in Table III. We compare only to the top performing models on the SALMon benchmark [35]. Flow-SLMs outperform TWIST and SpiritLM models in terms of acoustic consistency and sentiment alignment, but perform slightly worse in terms of background alignment. The poorer results on background alignment may be caused by the fact that we train our models mainly on MLS, which contains only audiobooks, and therefore is not exposed to a wide variety of acoustic backgrounds, whereas TWIST and SpiritLM are.

In terms of speaker similarity, we can see from Table II that, when TWIST and SpiritLM do not have access to a conditional vocoder (conditioning on the prompt speaker), they fail to maintain speaker consistency for the continuation. Flow-SLMs, on the other hand, are better able to retain speaker similarity. In terms of e2v FSD and wlm FSD (Table II), Flow-

TABLE IV
ABLATION STUDY ON THE DEV SET OF ZEROSPEECH AND VALIDATION LOSS ON THE LIBRISPEECH DEV SET. "TOKEN INPUT" MEANS THE MODEL TAKES DISCRETE INPUT, AND "VECTOR" INPUT MEANS THE MODEL TAKES CONTINUOUS VECTOR INPUT. L_{sem-k} REFERS TO THE LOSS FOR PREDICTING k FUTURE TOKENS. L_{CFM-N} INDICATES THAT THE CFM HEAD CONDITIONS ON N PREDICTED FUTURE TOKENS.

#	input	Objective	sWUGGY↑	sBLIMP↑	CE loss
TWIST-350M			70.2	55.2	
1	token	L_{sem-1}	69.3	55.7	3.44
2	token	L_{sem-2}	70.0	57.1	4.16
3	token	L_{sem-4}	63.7	53.3	4.94
4	vector	L_{sem-1}	63.7	52.4	2.28
5	vector	L_{sem-2}	68.1	55.5	3.17
6	vector	L_{sem-4}	68.6	57.8	4.27
7	vector	L_{sem-1} + L_{CFM-1}	67.5	57.0	4.27
8	vector	L_{sem-4} + L_{CFM-4}	68.3	57.1	4.23

SLMs again outperform SpiritLM and TWIST.

B. Does detailed acoustic context hurt semantic learning?

We conduct an ablation study on the dev set of the ZeroSpeech benchmark [34] and the LibriSpeech dev set to investigate whether using fine-grained acoustic context (using vector x instead of token z) compromises semantic understanding. As shown in row 1 of Table IV, using standard CE loss (L_{sem-1}) and Mimi semantic tokens as input produces comparable performance on sWUGGY and sBLIMP to TWIST-350M [1]. However, when we replace the input with continuous vectors (row 4), performance drops despite achieving a lower CE validation loss than row 1. This indicates that the model becomes better at predicting the next semantic token but fails to capture long-term dependencies, which are crucial for learning the lexicon, higher-level semantics, and syntax. We attribute this finding to the continuity of speech signals across time: When the context includes both semantic and acoustic information, the model can exploit acoustic continuity to predict the next semantic token easily.

To address this issue, we have proposed predicting additional future tokens. For example, using the L_{sem-4} objective to predict four future tokens (row 6) improves performance on both sWUGGY and sBLIMP. Predicting more future tokens helps in both token and vector input settings, but provides more benefit with vector inputs, as it mitigates their tendency to account only for local continuity. On the other hand, even predicting two future tokens helps the token-only model better capture lexical and grammatical patterns. This finding is an interesting by-product of our proposed future prediction loss, which may be interesting to study even in token-only models.

From the genPPL results in Table II, we observe that predicting further into the future and conditioning on future tokens improves generation perplexity (genPPL) significantly. Conversely, models that do not use semantic tokens tend to focus solely on local information, leading to poor genPPL.

C. Does the CFM prediction head hurt semantic learning?

When adding a CFM head to the model (row 7 in Table IV), we observe a drop in semantic metrics compared to row 6. This suggests that the model must encode additional acoustic information into the context vector to learn to generate acoustic details, which in turn reduces its capacity to retain semantic information. However, providing more conditioning (row 8) helps to mitigate this issue. Therefore, we adopt $L_{sem-4} + L_{CFM-4}$ as the default objective for Flow-SLM.

This performance drop is consistent with the findings of [16], where SpiritLM-expressive, which includes prosody and style tokens, performs worse than SpiritLM-base on semantic metrics (as shown also in Table I and Table II).

D. Is acoustic information degraded when predicting future semantic tokens?

The analysis above shows that predicting semantic tokens further into the future helps to retain semantic information. The next question we ask is whether we lose acoustic information when emphasizing semantic information in this way.

From the speaker preservation results in Table II, we see that when we learn solely from the CFM head (Flow-SLM-270M - future prediction - sem. token), the model has better speaker similarity compared to the models that use semantic tokens. For e2v FSD, which is used to evaluate expressiveness, we find that Flow-SLM works better with future prediction. We suspect that this is because expressiveness is tied to the semantic content. On the other hand, the results for wlm FSD, which focuses on the frame distribution similarity alone, show that the generated samples are closer to the ground truth distribution without future prediction.

VI. CONCLUSION AND FUTURE WORK

Our proposed Flow-SLM models jointly learn linguistic and acoustic information for textless spoken language models. Unlike prior approaches that require separate conditional vocoders, Flow-SLMs can natively generate acoustically consistent and expressive speech, largely without compromising semantic content. Our results show that the proposed approach of predicting multiple future tokens helps the model better capture linguistic content. Overall, our models perform competitively with existing models by many metrics, including both acoustic and linguistic evaluations, while using much less training data and compute, although there is a tradeoff between acoustic detail and semantic content. Future work on scaling the models up is needed to establish whether scale can close the remaining gaps in terms of semantic modeling.

Our work thus far focuses on the textless setting, which generally performs worse at learning semantics compared to models that generate text as intermediate representations [6]. Extending our model to a joint speech and text setup could therefore lead to more coherent generated speech. In terms of acoustic quality, it will be important to compare Flow-SLMs with models that generate acoustic RVQ tokens to better understand the trade-offs[6] and to extend the training data to a broader range of acoustic conditions.

[6]To the best of our knowledge, there is no publicly available textless SLM based on RVQ tokens.

REFERENCES

[1] M. Hassid, T. Remez, T. A. Nguyen, I. Gat, A. Conneau, F. Kreuk, J. Copet, A. Defossez, G. Synnaeve, E. Dupoux, R. Schwartz, and Y. Adi, "Textually pretrained speech language models," *Proc. NeurIPS*, 2023.

[2] K. Lakhotia, E. Kharitonov, W.-N. Hsu, Y. Adi, A. Polyak, B. Bolte, T.-A. Nguyen, J. Copet, A. Baevski, A. Mohamed, and E. Dupoux, "On generative spoken language modeling from raw audio," *Transactions of the Association for Computational Linguistics*, 2021.

[3] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, D. Roblek, O. Teboul, D. Grangier, M. Tagliasacchi, and N. Zeghidour, "AudioLM: A language modeling approach to audio generation," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 31, pp. 2523–2533, 2023.

[4] E. Kharitonov, A. Lee, A. Polyak, Y. Adi, J. Copet, K. Lakhotia, T.-A. Nguyen, M. Rivière, A. Mohamed, E. Dupoux, and W.-N. Hsu, "Text-free prosody-aware generative spoken language modeling," *Proc. ACL*, 2022.

[5] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "SoundStream: An end-to-end neural audio codec," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 30, pp. 495–507, 2021.

[6] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, "Moshi: A speech-text foundation model for real-time dialogue," *arXiv preprint arXiv:2410.00037*, 2024.

[7] X. Zhang, D. Zhang, S. Li, Y. Zhou, and X. Qiu, "SpeechTokenizer: Unified speech tokenizer for speech language models," in *Proc. ICLR*, 2024.

[8] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," *Proc. ICLR*, 2023.

[9] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 29, pp. 3451–3460, 2021.

[10] A. Polyak, Y. Adi, J. Copet, E. Kharitonov, K. Lakhotia, W.-N. Hsu, A. Mohamed, and E. Dupoux, "Speech resynthesis from discrete disentangled self-supervised representations," *Proc. Interspeech*, 2021.

[11] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.

[12] S. Arora, K.-W. Chang, C.-M. Chien, Y. Peng, H. Wu, Y. Adi, E. Dupoux, H.-Y. Lee, K. Livescu, and S. Watanabe, "On the landscape of spoken language models: A comprehensive survey," *arXiv preprint arXiv:2504.08528*, 2025.

[13] C. Tang, W. Yu, G. Sun, X. Chen, T. Tan, W. Li, L. Lu, Z. Ma, and C. Zhang, "SALMONN: Towards generic hearing abilities for large language models," *Proc. ICLR*, 2024.

[14] Y. Chu, J. Xu, X. Zhou, Q. Yang, S. Zhang, Z. Yan, C. Zhou, and J. Zhou, "Qwen-Audio: Advancing universal audio understanding via unified large-scale audio-language models," *arXiv preprint arXiv:2311.07919*, 2023.

[15] J.-C. Chou, C.-M. Chien, W.-N. Hsu, K. Livescu, A. Babu, A. Conneau, A. Baevski, and M. Auli, "Toward joint language modeling for speech units and text," *Findings of EMNLP*, 2023.

[16] T. A. Nguyen, B. Muller, B. Yu, M. R. Costa-jussa, M. Elbayad, S. Popuri, C. Ropers, P.-A. Duquenne, R. Algayres, R. Mavlyutov, I. Gat, M. Williamson, G. Synnaeve, J. Pino, B. Sagot, and E. Dupoux, "SpiritLM: Interleaved spoken and written language model," *Transactions of the Association for Computational Linguistics*, 2025.

[17] Z. Xie and C. Wu, "Mini-Omni: Language models can hear, talk while thinking in streaming," *arXiv preprint arXiv:2408.16725*, 2024.

[18] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," *Proc. ICLR*, 2023.

[19] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," *Proc. NeurIPS*, 2019.

[20] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Proc. NeurIPS*, 2020.

[21] S. Mehta, R. Tu, J. Beskow, Éva Székely, and G. E. Henter, "Matcha-TTS: A fast TTS architecture with conditional flow matching," in *Proc. ICASSP*, 2024.

[22] A. H. Liu, S. gil Lee, C.-H. H. Yang, Y. Gong, Y.-C. F. Wang, J. R. Glass, R. Valle, and B. Catanzaro, "UniWav: Towards unified pre-training for speech representation learning and generation," *Proc. ICLR*, 2025.

[23] A. H. Liu, M. Le, A. Vyas, B. Shi, A. Tjandra, and W.-N. Hsu, "Generative pre-training for speech with flow matching," *Proc. ICLR*, 2024.

[24] M. Le, A. Vyas, B. Shi, B. Karrer, L. Sari, R. Moritz, M. Williamson, V. Manohar, Y. Adi, J. Mahadeokar, and W.-N. Hsu, "Voicebox: Text-guided multilingual universal speech generation at scale," *Proc. NeurIPS*, 2023.

[25] Z. Yuan, Y. Liu, S. Liu, and S. Zhao, "Continuous speech tokens makes LLMs robust multi-modality learners," *arXiv preprint arXiv:2412.04917*, 2024.

[26] V. Pratap, Q. Xu, A. Sriram, G. Synnaeve, and R. Collobert, "MLS: A large-scale multilingual dataset for speech research," *Proc. Interspeech*, 2020.

[27] D. Galvez, G. Diamos, J. Ciro, J. F. Cerón, K. Achorn, A. Gopi, D. Kanter, M. Lam, M. Mazumder, and V. J. Reddi, "The People's Speech: A large-scale diverse English speech recognition dataset for commercial usage," *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[28] S. Mehta, M. H. Sekhavat, Q. Cao, M. Horton, Y. Jin, C. Sun, I. Mirzadeh, M. Najibi, D. Belenko, P. Zatloukal, and M. Rastegari, "OpenELM: An efficient language model family with open-source training and inference framework," *arXiv preprint arXiv:2404.14619*, 2024.

[29] T. Li, Y. Tian, H. Li, M. Deng, and K. He, "Autoregressive image generation without vector quantization," *Proc. NeurIPS*, 2024.

[30] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," *Proc. ICLR*, 2022.

[31] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *NeurIPS Workshop on Deep Generative Models and Downstream*, 2021.

[32] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," *Proc. ICLR*, 2020.

[33] J. Hwang, M. Hira, C. Chen, X. Zhang, Z. Ni, G. Sun, P. Ma, R. Huang, V. Pratap, Y. Zhang, A. Kumar, C.-Y. Yu, C. Zhu, C. Liu, J. Kahn, M. Ravanelli, P. Sun, S. Watanabe, Y. Shi, Y. Tao, R. Scheibler, S. Cornell, S. Kim, and S. Petridis, "TorchAudio 2.1: Advancing speech recognition, self-supervised learning, and audio processing components for PyTorch," in *Proc. ASRU*, 2023.

[34] T. A. Nguyen, M. de Seyssel, P. Rozé, M. Rivière, E. Kharitonov, A. Baevski, E. Dunbar, and E. Dupoux, "The Zero Resource Speech Benchmark 2021: Metrics and baselines for unsupervised spoken language modeling," *NeurIPS Workshop on Self-Supervised Learning for Speech and Audio Processing*, 2020.

[35] G. Maimon, A. Roth, and Y. Adi, "SALMon: A suite for acoustic language model evaluation," in *Proc. ICASSP*, 2025.

[36] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR corpus based on public domain audio books," in *Proc. ICASSP*, 2015.

[37] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.

[38] A. Grattafiori *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[39] G. Maimon, A. Elmakies, and Y. Adi, "Slamming: Training a speech language model on one GPU in a day," *Findings of ACL*, 2025.

[40] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, J. Wu, L. Zhou, S. Ren, Y. Qian, Y. Qian, J. Wu, M. Zeng, X. Yu, and F. Wei, "WavLM: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, 2022.

[41] H. He, Z. Shang, C. Wang, X. Li, Y. Gu, H. Hua, L. Liu, C. Yang, J. Li, P. Shi, Y. Wang, K. Chen, P. Zhang, and Z. Wu, "Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation," in *Proc. SLT*, 2024.

[42] Z. Ma, Z. Zheng, J. Ye, J. Li, Z. Gao, S. Zhang, and X. Chen, "emotion2vec: Self-supervised pre-training for speech emotion representation," *Findings of ACL*, 2024.

[43] M. Baas, B. van Niekerk, and H. Kamper, "Voice conversion with just nearest neighbors," *Proc. Interspeech*, 2023.

[44] S. Maiti, Y. Peng, S. Choi, J. weon Jung, X. Chang, and S. Watanabe, "VoxtLM: Unified decoder-only models for consolidating speech recognition, synthesis and speech, text continuation tasks," in *Proc. ICASSP*, 2024.

[45] J. Hoffmann *et al.*, "An empirical analysis of compute-optimal large language model training," *Proc. NeurIPS*, 2022.